BOSTON UNIVERSITY

EC 327 – Introduction to Software Engineering

Week 1 and 2

Course Overview

Six Level Computer

Simplified Assembly

Prof. Douglas Densmore

Select material courtesy of Prof. Ayse Coskun and Prof. Ari Trachtenberg

First Things First

- Are you enrolled? If not, resolve ASAP! (53 registered, 105 spots!)
  - Brief history of the course enrollments – 40 ->60 ->80->100+**->72 (COVID??)->107 (2022)->52??**
    - Wow! This is good. You will get more attention!!!
- AUDITING THE CLASS. Can have access to blackboard. Seating priority given to those enrolled.
  - EC 327 offered EVERY semester

BOSTON UNIVERSITY

What is this class about?

- ***Software?***
  - Why have software?
  - What goes into making software work?
  - How do we break it? Test it? Verify it? Is there a difference?

- ***Hardware?***
  - Why would you need to know HW?
  - Why is hardware “hardware” and not “software”? What is the difference?

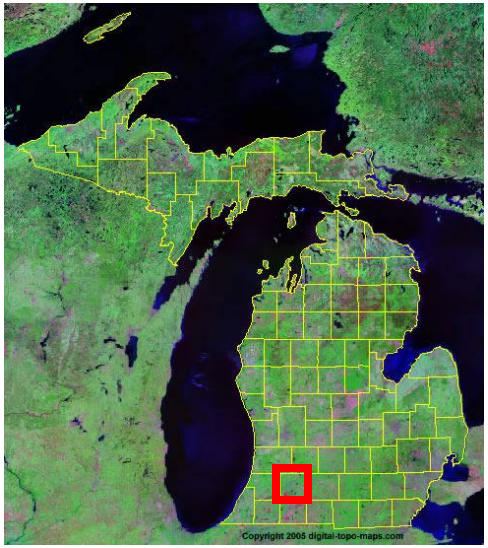
Computer Engineering concepts

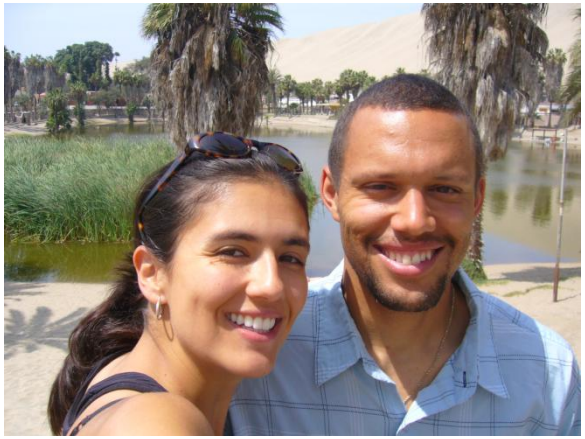
- What is abstraction?
- What is modularity?
- What is computation vs. communication vs. coordination
- What is a computer?
- What is hardware?
- What is software?
- What is functionality vs. architecture?
- What is behavior vs. performance?

Relationship to other ECE courses?

- How does EC327 relate to?
  - EC 311 (Logic Design), 413 (Comp Org), 440 (OS)
  - EK 307 (Circuit Theory)
  - EC 330 (Algorithms)
  - EC 450 (Microprocessors)
  - EC 535 (Embedded Comp), 551 (FPGAs), 571 (VLSI)

Who's teaching this course??



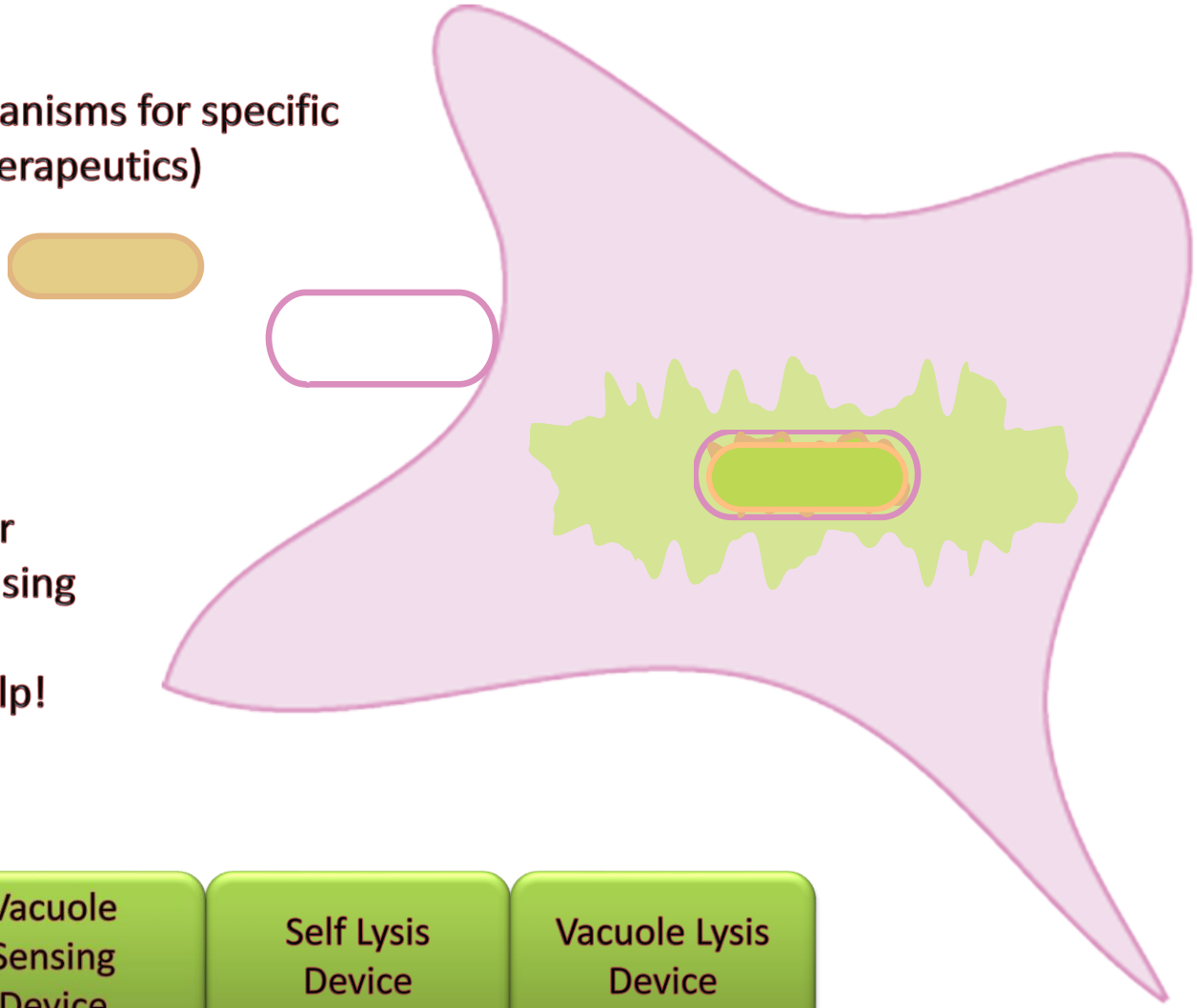


MICHIGAN XILINX CYPRESS intel Cal BOSTON UNIVERSITY

BOSTON UNIVERSITY

What do I do? - General

Synthetic Biology:
Engineering living organisms for specific tasks (example bio-therapeutics)



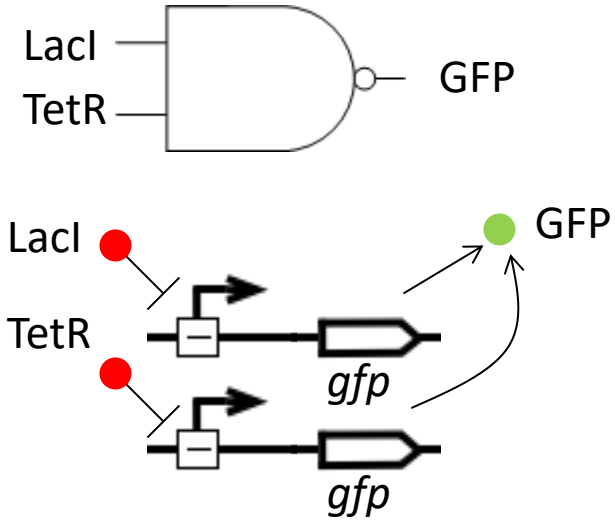
How do we engineer bacteria to do this using "standard biological parts"? EECS can help!

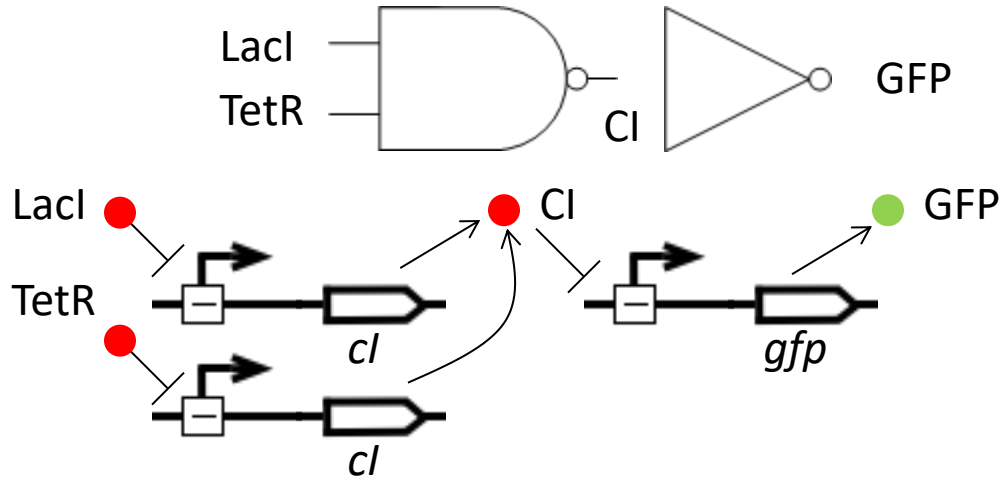
Invasion Device	Vacuole Sensing Device	Self Lysis Device	Vacuole Lysis Device

Courtesy of: Josh Kittleson – UC Berkeley

BOSTON UNIVERSITY

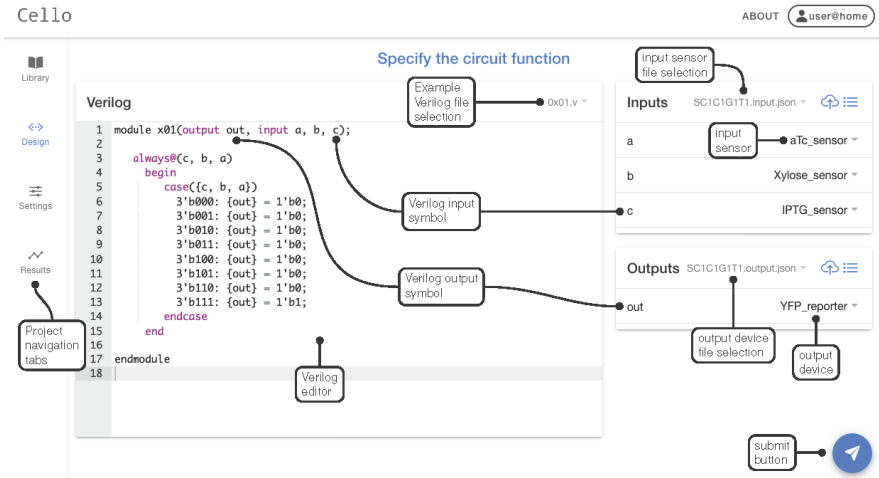
Why am I in the ECE dept?



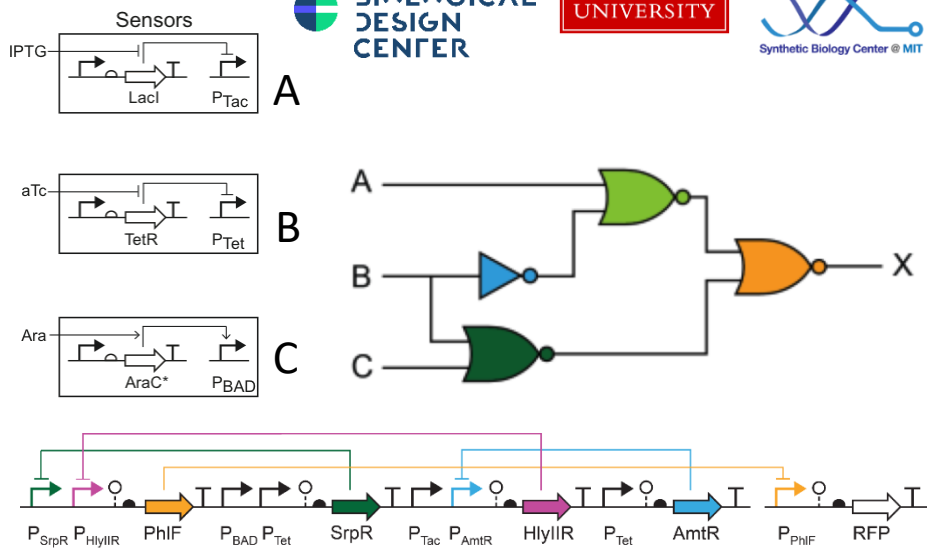


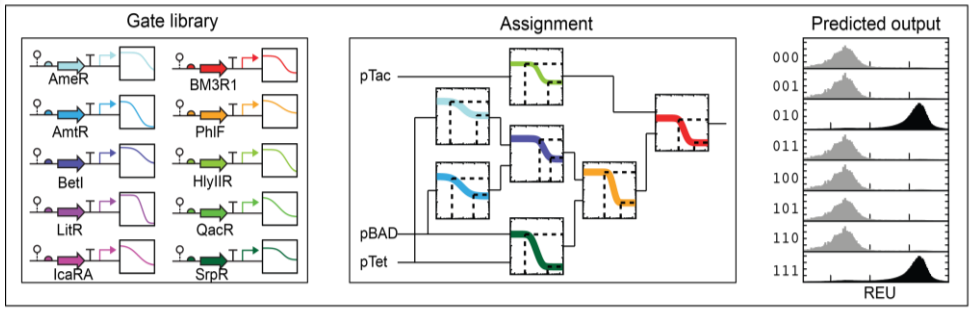
BOSTON UNIVERSITY

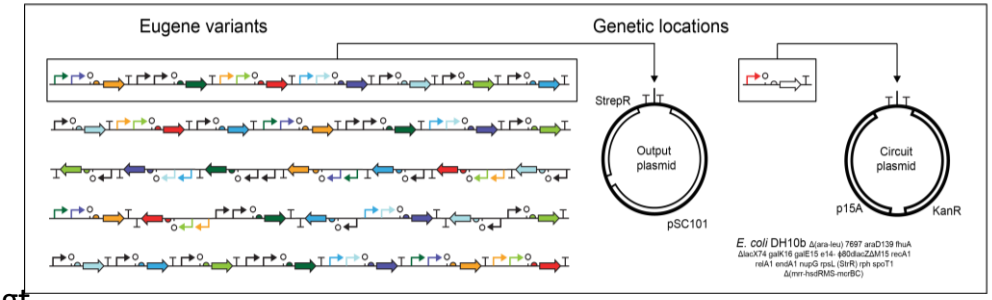
CELLO www.cellocad.org



Collaboration with Chris Voigt, MIT







Timothy Jones, Chris Myers, Christopher A. Voigt, Douglas Densmore, *Genetic Circuit Design Automation with Cello 2.0*, **Nature Protocols.**.

Ye Chen, Shuyi Zhang, Eric M. Young, Timothy S. Jones, Douglas Densmore, Christopher A. Voigt, *Genetic Circuit Design Automation for Yeast*, **Nature Microbiology**, 2020.

Alec Nielsen, Bryan Der, Jonghyeon Shin, Prashant Vaidyanathan, Vanya Paralanov, Elizabeth Strychalski, David Ross, Douglas Densmore, and Christopher A. Voigt, *Genetic Circuit Design Automation*, **Science**, 2016.

Course Philosophy
(aka “this course is hard but curved”)

Historical Averages Over 9 Semesters

24%	A
14%	A-
20%	B+
16%	B
10%	B-
4%	C+
3%	C
1%	C-
0%	D+
3%	D
0%	D-
2%	F
4%	W/I

84% get an A or B

Procrastinate
Don’t communicate
Don’t turn in things
Cheat

BOSTON UNIVERSITY

Course logistics

- **Lab Instructor:** Each lab has a “lead” (Friday has two TFs)
- **(U)TFs:** Vugar Amirov, Vincent Zhang, Yoan Zhao
- **GTFs:** Pravi Samaratunga
- Review Course Info, Course Topics, Course Schedule, **Reading List**
- Blackboard site is key for assignments
- Piazza main source of help/questions!!
- **Using GitHub** for Programming Assignments

BOSTON UNIVERSITY

“New-ish” for Spring 2026

- UTFs and GTFs paired up with students
  - Mentorship, grading, code reviews, OH, etc.
  - See Blackboard for pairing list (red means you are NOT in their lab)
- Programming Assignments will be submitted using GitHub (more to come)
- **Autograding**
- Project does NOT have to be an Android app
  - Must meet technical and SW engineering criteria
  - **Liberal use of AI allowed**
- New-ish Textbook (clearer examples, better illustrations)
- Gradescope?? TBD

BOSTON UNIVERSITY

Note taking (Extra)

- Send notes to us (specifically Pravi)
  - Counts toward class participation
  - Note quality will be taken into consideration (Don't send gibberish)
- Credited notes will be put on Blackboard
- Must come within 24 hours of the lecture to get "credit"
  - Feel free to send in notes later however
- Subject: <Username>_Notes_MMDDYY

BOSTON UNIVERSITY

What is SW engineering?

- "Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software." @ wiki.
  - First reading assignment!

- SW abstraction dates back to the 1940s.

What do these have to do with SW engineering?

Tools

- Modeling
- Simulation
- Experiments
- Analysis

Deliverables

- User interfaces
- Algorithms/methods
- Architectures
- Systems

Where does SW go today?

PCs, supercomputers, data centers, …

Transportation

Communication, mobile apps

Office Equipments

Avionics

Consumer Electronics

Medical Instruments

Household Appliances

BOSTON UNIVERSITY

Growth of Software

Microsoft Windows

Lines of code (LoC)

Ship Date	Product	Dev Team Size	Test Team Size	Lines of code
Jul-93	NT 1.0 (released as 3.1)	200	140	4-5 million
Sep-94	NT 2.0 (released as 3.5)	300	230	7-8 million
May-95	NT 3.0 (released as 3.51)	450	325	9-10 million
Jul-96	NT 4.0 (released as 4.0)	800	700	11-12 million
Dec-99	NT 5.0 (Windows 2000)	1,400	1,700	29+ million
Oct-01	NT 5.1 (Windows XP)	1,800	2,200	40 million
Apr-03	NT 5.2 (Windows Server 2003)	2,000	2,400	50 million
	Vista			> 50 million

- Standard : 1 error / 1000 lines of professional code

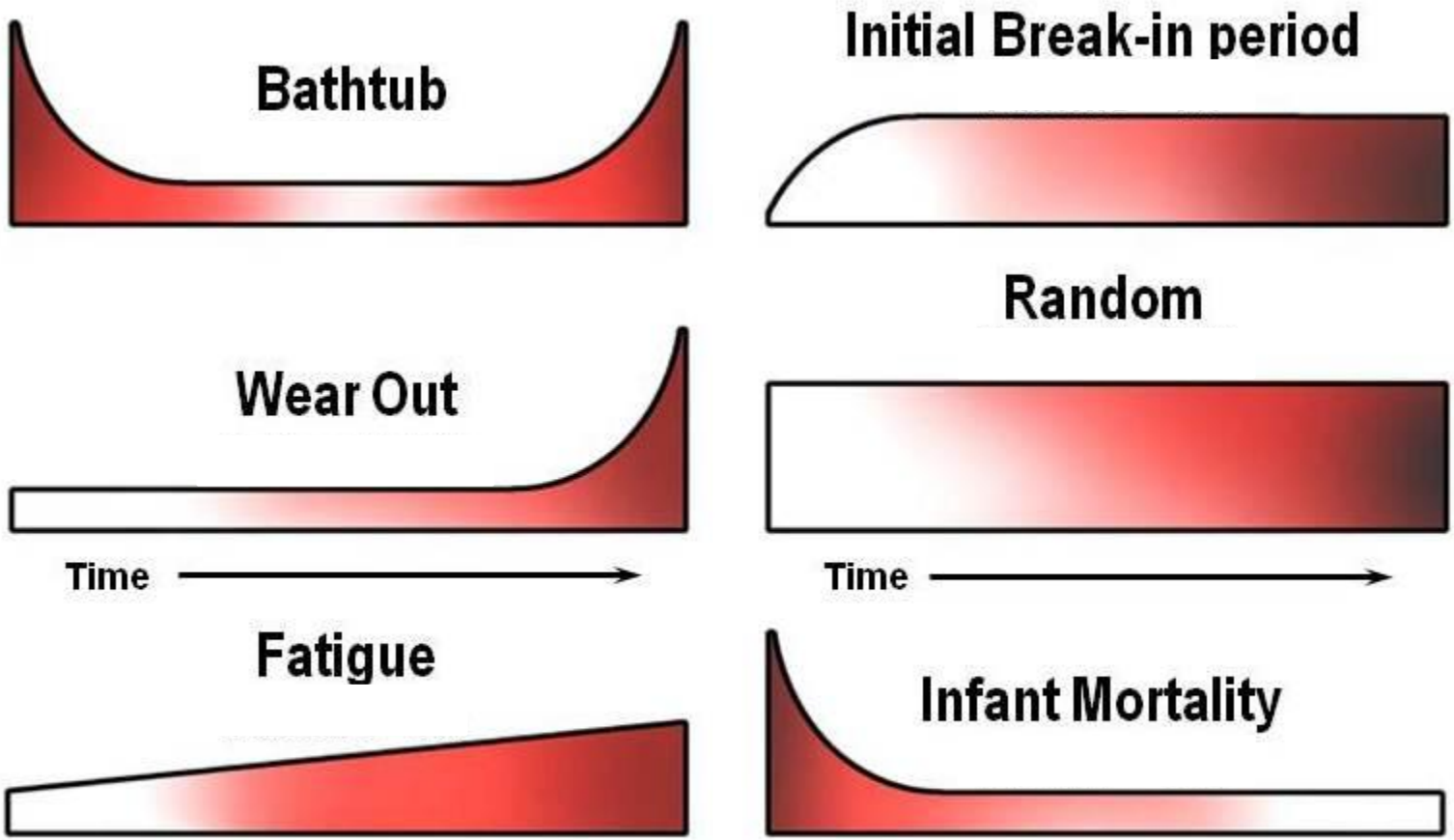
BOSTON UNIVERSITY

What can go wrong?

- Some examples:
  - Mars orbiter crash in 1999
  - Therac 25 (radiation therapy machine) cancer deaths in mid 80s
  - Mars Pathfinder reset problem 1997
  - Voting machine problems (2004, 2008, many others)
  - Y2K bug
  - Toyota Prius call-back in 2005
- Interested in more?
  - Google search, or
  - **History's Worst Software Bugs:** http://www.wired.com/software/coolapps/news/2005/11/69355

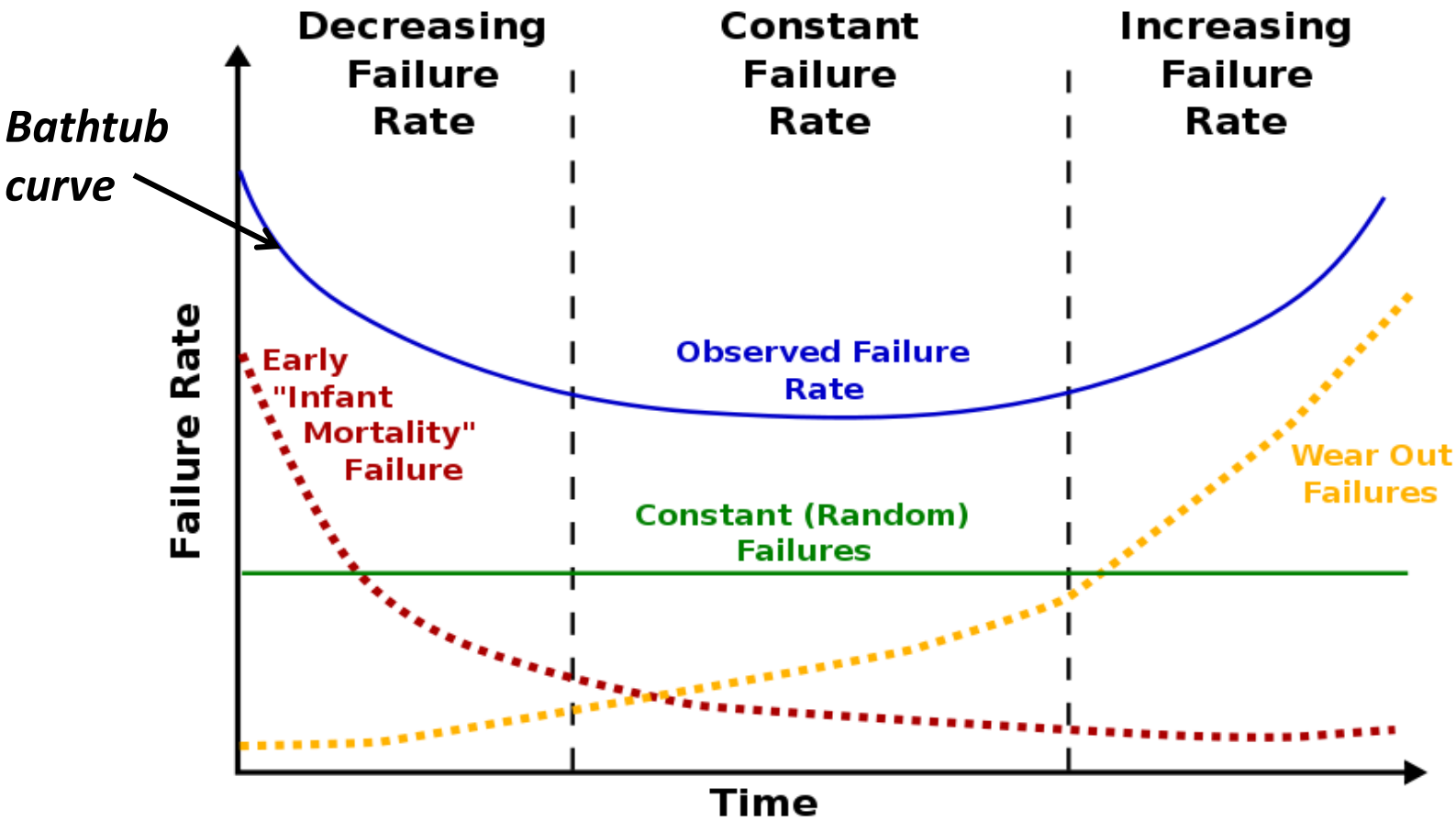
BOSTON UNIVERSITY

Failure curves - HW



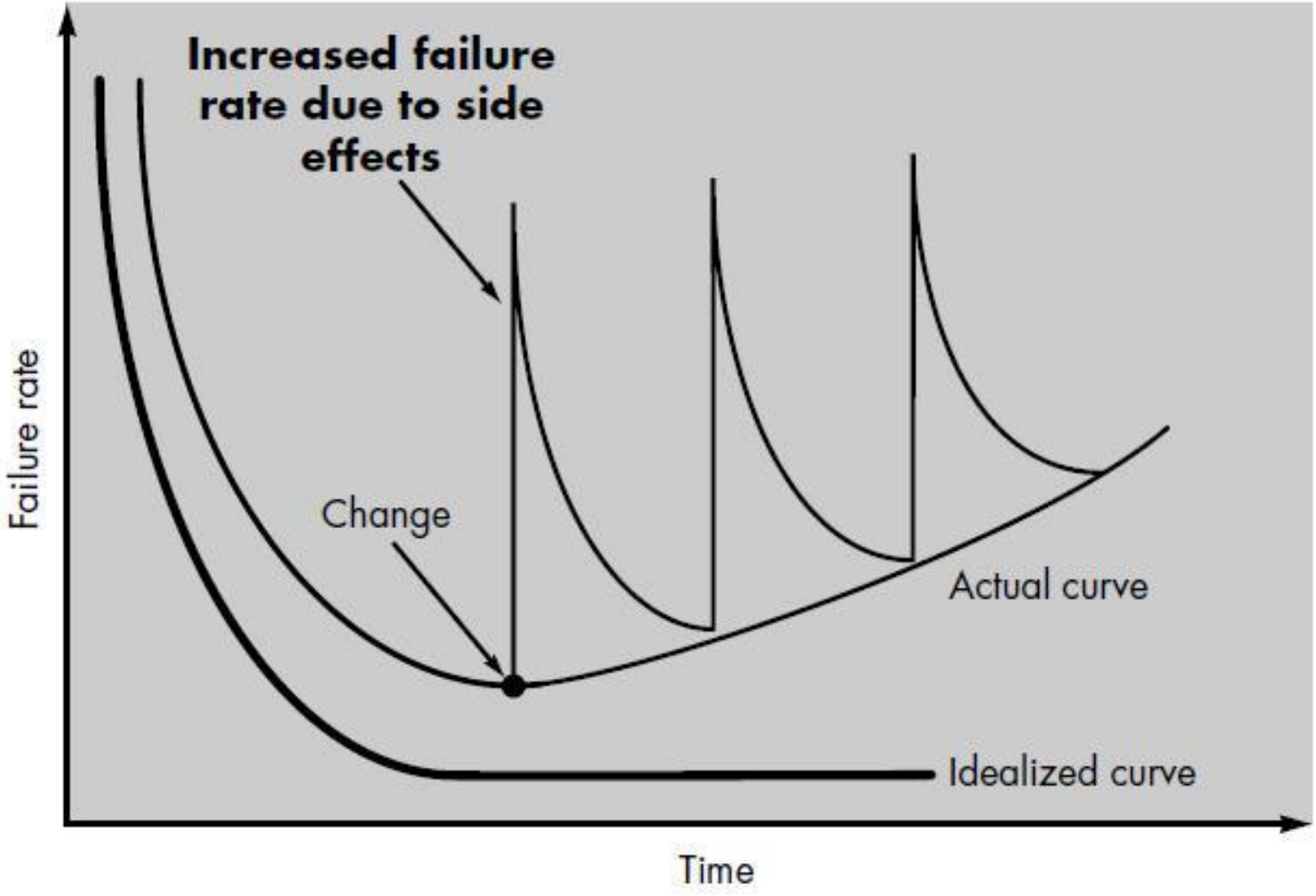
BOSTON UNIVERSITY

Failure curves - HW



BOSTON UNIVERSITY

Failure curves - SW



BOSTON UNIVERSITY

SW engineering in the market

- Drop in CE/CS majors
  - Economy? Out-sourcing? End of dot-com bubble?
- La
  -
- C
  -

AI - TBD

- Future:
  - 70% of all future jobs are projected to be in computer science related topics

BOSTON UNIVERSITY

Again, what is this class about?

- *Goal:* Building up the necessary competence and analytical skills for designing high quality SW
- Bottom-up approach
  - Start from the computer fundamentals, machine language, compilers
  - Build up to high-level languages, basic data structures, algorithms
- What next?
  - Algorithms, operating systems, advanced data structures, …

Break – Class 1

BOSTON UNIVERSITY

Six Level Computer

- Digital Logic (Level 1)
- Micro-architecture (Level 2)
- Instruction Set Architecture (ISA) (Level 3)
- Operating System (OS) (Level 4)
- Assembly Language (Level 5)
- High Level Language (Level 6)

Other Levels?

- Why levels?
- Pros?
- Cons?
- CISC vs. RISC

5 Classic Components of a Computer

- Patterson/Hennessey

1. Control
2. Datapath
3. Memory
4. Input
5. Output

BOSTON UNIVERSITY

Languages

- COBOL (Common Business Oriented Language)
- FORTRAN (FORmula TRANslation)
- BASIC (Beginner's All-purpose Symbolic Instruction Code)
- Pascal (named after Blaise Pascal)
- Ada (named for Ada Lovelace)
- C, C++, C#
- Java
- Python, Perl

Interpreted vs. Compiled Languages

- ?
- Examples?
- Pros vs. Cons?

Role of the OS

- Control and monitor system activities
  - What activities?
- Allocate and assign system resources
  - What resources?
- Schedule operations
  - What operations?

- What are some Operating Systems?

Overall Design Flow

1. Source Code
2. Compiler
3. Assembly Code
4. Assembler
5. Machine Code
6. Linker
7. Executable
8. Loader

FAQ

- “I am not really a software person, I am more of a hardware guy. Do I really need to learn software?”
- “Linux environment seems really really primitive. Why do I need to learn this, can’t I just use Windows and be happy?”
- “I am very busy; can you just tell me when the pop quizzes are so I can make sure to show up then?”

FAQ (cont'd)

- "I keep submitting everything on time and I work really hard. I noticed there are some other students in class who are submitting their labs late. How do I know I will get a better grade than those guys?"

- "If I do 60% of the labs on time and turn in 20% of them slightly late and the rest really late, do one third of the homework perfectly and the rest with some mistakes, miss one quiz but ace the other two, have a midterm grade on the average but a great score for the final, do you think I'll get a B+?"

BOSTON UNIVERSITY

FAQ (cont'd)

- "I am a sophomore, and I never took a programming class before. What do I do now?"

- "I just arrived to class, what did I miss?"

Solving A Problem

- Four people come to a river at night. There is a bridge they need to cross, but the bridge can hold two people at most at a time. They have one torch and the torch has to be used while crossing the bridge. Person A can cross the bridge in 1 minute, B in 2 minutes, C in 5 minutes, and D in 10 minutes. When two people cross the bridge together, they must move at the slower person's pace. Can they all cross the bridge in 17 minutes (or less)?

- Given four people and an arbitrary minimum crossing time for each person, can you come up with a strategy to compute the minimum time needed for crossing the bridge?
  - How many “steps” do you need in your algorithm?
  - What information do you need to store?
  - How do you verify that your algorithm is correct?

BOSTON UNIVERSITY

Next Time

- Six-level computer
  - **See reading list**

- Reading (links on Blackboard -> Content -> Reading Material)
  - SW engineering @ Wiki
  - Patterson's IEEE Spectrum article on current challenges in programming
  - Wired article (Biologists and Programing), NYT (Plagiarism)
- Other action items:
  - Check lab access (PHO 307, 305?, 115?, 117?)
  - Check that you have Blackboard access
  - Check that you have Piazza access
  - (Next week) Getting started on Unix/Linux

* Check Blackboard for announcements!

BOSTON UNIVERSITY

Start – Class 2

BOSTON UNIVERSITY

Talk about “Bridge Problem”

BOSTON UNIVERSITY

Simplified Assembly Part 1

- Outline
  - Describe the architecture
  - Hypothesize on a microarchitecture
  - Go over ISA and document
  - Manual Assembler example and execution

Break – Class 2

BOSTON UNIVERSITY

Simplified Assembly Part 2

- Outline
  - Demonstrate the assembler (python script). How does it work?
  - Demonstrate the simulator. How does it work?
  - 2 in-class **SIMPLE** example programs with worksheet.

Next Time

- More complex assembly examples using our class ISA
- Disassemblers
- Examples of other ISAs
  - Pros, Cons, Performance

- Play with the simulator and practice a bit
  - See tutorial documents on Blackboard
- HW 1 – Posted Wed 1/28 (due 2/18) – Start EARLY.

* Check Blackboard for announcements!